11조 RISC-V 시뮬레이터 - 개인 이해도 평가(기말고사 프로젝트 문항) 대비 정리

> **주의 1 - 이 파일은 제출물이 아니고 시험장에 가져갈 수 없다.** 명세 9절: 시험장에는 "제출 마감 시 고정된 본인 팀 소스 코드를 인쇄한 자료"만 열람할 수 있고, 별도 해설·예상 답안·추가 메모는 허용되지 않는다. 그래서 이 파일은 `riscv-sim/`(제출 폴더) 밖에 두었다.
>
> **주의 2 - 줄 번호는 현재 소스 기준이다.** 제출 전에 코드를 고치면 줄 번호가 바뀐다. 최종 제출본으로 `python study/build_pdfs.py`를 다시 실행해 인쇄본과 이 문서를 맞춘다.

---

0. 시험 형식 (명세 9·10절)

- 프로젝트 100점 = **팀 결과물 60점 + 개인 이해도 40점.** 개인 점수는 기말고사 시간의 프로젝트 문항으로 매긴다.
- AI·인터넷·전자기기·상호 협의는 금지다. **팀 소스 인쇄본만** 볼 수 있다(제출본과 같아야 함). 팀원끼리도 독립적으로 답안을 쓴다.
- 문항 유형: ① 자기 코드·보고서 설명 ② 실행 결과 예측 ③ 오류 진단 ④ 테스트 설계 ⑤ 작은 변경의 영향 분석
- "같은 공통 질문이라도 **팀의 구현에 따라 답이 달라질 수 있다**" → 일반론이 아니라 **우리 코드의 파일·함수·줄**로 답해야 한다.
- 시험에는 **새로운 입력·검증 상황**이 나온다. 외우기보다 손으로 해독하고 실행하는 능력이 필요하다.

답안 작성 공식: ① 우리 코드 위치(파일:함수:줄) → ② 그 코드가 하는 일 → ③ 주어진 입력에 적용한 계산 과정 → ④ 결론(값·분기 여부·출력)

---

1. 반드시 외울 숫자와 규칙

항목	값
초기 PC = text 시작	`0x00040000`
data 시작	`0x10000000`, 크기 1,024 B (0x400) → 마지막 바이트 `0x100003FF`, 마지막 워드 `0x100003FC`
레지스터	x0~x31, 32비트, 모두 0으로 시작, x0은 항상 0, 스택 포인터 초기화 없음
바이트 순서	little-endian: `b0 + (b1<<8) + (b2<<16) + (b3<<24)`, `78 56 34 12` → `0x12345678`
종료	지원하지 않는 명령어를 fetch하면 **실행하지 않고** 종료. 출력 PC는 **fetch한 주소**(PC+4 아님)
표준 출력	`PC 0x%08x invalid instruction 0x%08x\n`: "PC" 대문자, 공백 1칸, 소문자 8자리, 줄바꿈
data.out	0x10000000부터 정확히 1,024 B, 바이너리, 덮어쓰기, invalid 이후 효과 없음
실행 한도	정상 명령어 최대 1,000,000개. 그다음 fetch가 invalid면 정상, 지원 명령어면 비정상 종료
입력	헤더 없는 바이너리. argv: text 경로, data 경로 (2개)
text.bin	4의 배수, 마지막 4바이트가 0x00000000
data.bin	0~1,024 B. 전체를 0으로 초기화한 뒤 적재

항목	값
lw/sw	주소는 4바이트 정렬, 데이터 영역 안. text 쓰기 불가
우리 종료 코드	0 정상 / 1 인수·파일 오류 / 2 실행 한도 초과 / 3 범위·정렬 위반

---

2. 우리 코드 지도 (인쇄본에서 펼 곳)

주제	파일:줄	핵심 코드
주소 상수	`rvsim/memory.py:13-15`	`TEXT_BASE = 0x00040000`, `DATA_BASE = 0x10000000`, `DATA_SIZE = 1024`
little-endian 읽기	`rvsim/memory.py:22-28` `read_word_le`	`return b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)`
little-endian 쓰기	`rvsim/memory.py:31-36` `write_word_le`	`buf[offset] = value & 0xFF` …
메모리 초기화·적재	`rvsim/memory.py:42-48` `Memory.__init__`	`self.text = bytes(...)`, `self.data = bytearray(DATA_SIZE)`, `self.data[0:n] = ...`
fetch	`rvsim/memory.py:50-56` `Memory.fetch`	범위·정렬 검사 → `read_word_le(self.text, offset)`
lw/sw 주소 검사	`rvsim/memory.py:58-72` `_data_offset`, `load_word`, `store_word`	`addr % 4 != 0 or offset < 0 or offset + 4 > DATA_SIZE`
입력 파일 읽기	`rvsim/memory.py:75-78` `read_binary_file`	`open(path, "rb")`
data.out 쓰기	`rvsim/memory.py:81-88` `write_data_out`	`open(path, "wb")`, `memory.data[0:DATA_SIZE]`
32비트 자르기	`rvsim/bits.py:16-22` `to_u32`	`value & 0xFFFFFFFF`
부호 해석	`rvsim/bits.py:25-33` `to_signed32`	bit31이 1이면 `value - 0x100000000`
부호 확장	`rvsim/bits.py:36-45` `sign_extend`	최상위 비트가 1이면 `value - (1 << bit_count)`
비트 필드	`rvsim/bits.py:48-53` `get_bits`	`(value >> lo) & ((1 << (hi-lo+1)) - 1)`
해독 표	`rvsim/decoder.py:42-51`	`R_TYPE_TABLE`, `OP_IMM_TABLE`, `LOAD_TABLE`, `STORE_TABLE`, `BRANCH_TABLE`
즉시값 I/S/B/U	`rvsim/decoder.py:71-99`	`imm_i`, `imm_s`, `imm_b`(91행 bit7 → imm[11], 94행 13비트 확장), `imm_u`
해독·invalid 판별	`rvsim/decoder.py:102-148` `decode`	필드 추출 108-113행, opcode별 분기, 표에 없으면 `return None`
레지스터·PC 초기값	`rvsim/cpu.py:21-25` `CPU.__init__`	`self.regs = [0] * 32`, `self.pc = TEXT_BASE`
실행 한도 상수	`rvsim/cpu.py:7`	`MAX_INSTRUCTIONS = 1000000`
x0 고정 + 32비트 저장	`rvsim/cpu.py:27-30` `write_reg`	`if index != 0: self.regs[index] = to_u32(value)`
실행 루프	`rvsim/cpu.py:32-47` `CPU.run`	39행 fetch_pc 보존, 40행 fetch, 41행 decode, **42행 invalid 확인 → 44행 한도 확인**, 46행 execute

주제	파일:줄	핵심 코드
명령어 실행	`rvsim/cpu.py:49-104` `CPU.execute`	55행 `next_pc = pc + 4`, 58~65 R형, 68~73 I형, 76~77 lui, 80~85 lw/sw, 88~99 분기, 104행 반환
blt / bge	`rvsim/cpu.py:94-99`	`if to_signed32(rs1) < to_signed32(rs2): next_pc = pc + imm`
인수·오류 처리	`sim.py:36-65` `main`	37행 인수 개수, 50~54행 크기 경고, 59행 `cpu.run()`, 60~65행 예외 → 종료 코드 2/3
종료 출력	`sim.py:27-29`, `sim.py:69`, `sim.py:75`	형식 문자열, data.out 저장, `sys.stdout.buffer.write(...)`

---

3. 실행 흐름 한 장 요약

```
sim.main
 ├ 인수 2개 확인 (37행)
 ├ read_binary_file("rb")로 text, data 읽기 (43~44행)
 ├ Memory(text, data): text=bytes, data=1024B 0으로 초기화 후 복사
 ├ CPU(memory): regs=[0]*32, pc=0x00040000, executed=0
 ├ cpu.run() 반복:
 │    fetch_pc = pc
 │    word = memory.fetch(fetch_pc)        ← little-endian 4바이트
 │    inst = decode(word)
 │    inst is None?  → return (fetch_pc, word)          [정상 종료]
 │    executed >= 1,000,000? → ExecutionLimitExceeded   [비정상 종료]
 │    pc = execute(inst, fetch_pc); executed += 1
 ├ write_data_out("data.out")  ("wb", 1024B)
 └ stdout.buffer.write("PC 0x%08x invalid instruction 0x%08x\n")
```

명세 예시 추적표 (`python riscv-sim/tools/trace.py`로 같은 표를 볼 수 있다)

#	PC	기계어	명령어	결과
1	0x00040000	0x100000b7	lui x1, 0x10000	x1 = 0x10000000
2	0x00040004	0x00700113	addi x2, x0, 7	x2 = 7
3	0x00040008	0x00500193	addi x3, x0, 5	x3 = 5
4	0x0004000c	0x00310233	add x4, x2, x3	x4 = 0xc
5	0x00040010	0x0040a023	sw x4, 0(x1)	[0x10000000] = 0c 00 00 00
-	0x00040014	0x00000000	invalid	실행 안 함 → `PC 0x00040014 invalid instruction 0x00000000`

---

4. 명령어 인코딩

4.1 필드 배치

```
비트       31      25 24  20 19  15 14 12 11        7 6     0
R형       | funct7   | rs2  | rs1  |funct3|    rd     |opcode|
I형       |   imm[11:0]     | rs1  |funct3|    rd     |opcode|
S형       | imm[11:5]| rs2  | rs1  |funct3| imm[4:0]  |opcode|
B형       |i[12|10:5]| rs2  | rs1  |funct3|i[4:1|11] |opcode|
U형       |          imm[31:12]          |    rd     |opcode|
```

- opcode는 항상 [6:0], rd는 [11:7], funct3는 [14:12], rs1은 [19:15], rs2는 [24:20], funct7은 [31:25]에 있다.
- S형과 B형은 rd 자리에 즉시값 일부가 들어간다(sw와 분기에는 rd가 없다).
- B형은 imm[0]이 항상 0이라 저장하지 않는다. 그래서 13비트 즉시값(-4096 ~ 4094)을 12개 비트로 표현한다.

4.2 opcode·funct 표 (우리 `decoder.py:42-51`)

opcode (16진)	종류	funct3 → 명령어
0110011 (0x33)	R	000+funct7 0000000 = add, 000+0100000 = sub, 111+0000000 = and, 110+0000000 = or
0010011 (0x13)	I 연산	000 = addi, 111 = andi, 110 = ori
0000011 (0x03)	load	010 = lw
0100011 (0x23)	store	010 = sw
1100011 (0x63)	branch	000 = beq, 001 = bne, 100 = blt, 101 = bge
0110111 (0x37)	U	lui

16진 → 2진 빠른 표: 0 0000, 1 0001, 2 0010, 3 0011, 4 0100, 5 0101, 6 0110, 7 0111, 8 1000, 9 1001, a 1010, b 1011, c 1100, d 1101, e 1110, f 1111

하위 바이트로 opcode 빨리 보기: 하위 7비트가 opcode다.

- 끝 두 자리 `33`, `b3` → 0x33 (R)
- `13`, `93` → 0x13 (I 연산)
- `03`, `83` → 0x03 (load)
- `23`, `a3` → 0x23 (store)
- `63`, `e3` → 0x63 (branch)
- `37`, `b7` → 0x37 (lui)

예: `0x00310233`의 끝 바이트 `0x33` → R형 / `0x100000b7`의 `0xb7` = 1011 0111, 하위 7비트 011 0111 = 0x37 → lui.

4.3 손으로 해독하기 - 예제 `0xfe0718e3`

1. 2진수로 바꾼다: `1111 1110 0000 0111 0001 1000 1110 0011`
2. opcode [6:0] = `110 0011` = 0x63 → **분기**
3. funct3 [14:12] = `001` → **bne**
4. rs1 [19:15] = `01110` = 14, rs2 [24:20] = `00000` = 0
5. 즉시값 비트를 모은다.

    - inst[31] = 1 → imm[12]
    - inst[30:25] = `111111` → imm[10:5]
    - inst[11:8] = `1000` → imm[4:1]
    - inst[7] = 1 → imm[11]
6. 13비트로 조립한다: imm[12..0] = `1 1 111111 1000 0` = `1111111110000` = 0x1FF0
7. 최상위 비트가 1이므로 음수다: 0x1FF0 - 0x2000 = **-16**
8. 결론: **`bne x14, x0, -16`** (분기하면 이 명령어 주소 - 16으로 간다)

예제 `0x00f12623`

- 끝 바이트 `0x23` → store, funct3 = `010` → sw
- rs2 = [24:20] = `01111` = 15, rs1 = [19:15] = `00010` = 2
- imm[11:5] = `0000000`, imm[4:0] = [11:7] = `01100` = 12
- 결론: **`sw x15, 12(x2)`**

4.4 손으로 인코딩하기 - 예제

`sw x8, -8(x2)`

1. imm = -8을 12비트로 쓰면 `1111 1111 1000`(0xFF8)이다.
2. imm[11:5] = `1111111`(0x7F), imm[4:0] = `11000`(0x18)
3. 조립: `(0x7F<<25) | (8<<20) | (2<<15) | (0b010<<12) | (0x18<<7) | 0x23`
4. = `0xFE000000 + 0x00800000 + 0x00010000 + 0x2000 + 0xC00 + 0x23` = **`0xfe812c23`**

`beq x1, x2, -12`

1. imm = -12를 13비트로 쓰면 `1 1111 1111 0100`이다.
2. imm[12] = 1, imm[11] = 1, imm[10:5] = 111111, imm[4:1] = 1010
3. 조립: `(1<<31) | (0x3F<<25) | (2<<20) | (1<<15) | (0<<12) | (0xA<<8) | (1<<7) | 0x63`
4. = **`0xfe208ae3`**

4.5 연습 (정답은 시뮬레이터로 확인함)

기계어	정답
0x00a00513	addi x10, x0, 10
0x40b50533	sub x10, x10, x11
0xffc4a303	lw x6, -4(x9)
0x0062c463	blt x5, x6, 8
0x800002b7	lui x5, 0x80000 (x5 = 0x80000000)
0x0003a283	lw x5, 0(x7)
0x00b57533	and x10, x10, x11
0xfff00093	addi x1, x0, -1
0x0003b283	**invalid** (load인데 funct3 = 011, RV64의 ld)
0x02b50533	**invalid** (R형, funct7 = 0000001 → mul)
0x0062e463	**invalid** (branch, funct3 = 110 → bltu)
0x40b57533	**invalid** (and의 funct3 = 111인데 funct7 = 0100000)

어셈블리	정답
addi x5, x6, -3	0xffd30293
lw x10, 16(x11)	0x0105a503
add x3, x1, x2 / sub x3, x1, x2	0x002081b3 / 0x402081b3 (funct7만 다름)
lui x7, 0x12346	0x123463b7
bge x0, x5, 20	0x00505a63
ori x4, x4, -2048	0x80026213
beq x1, x2, 2048	0x002080e3 (imm[11] = 1이 **bit 7**로 간다 → 하위 바이트 `e3`)

---

5. 32비트와 부호

5.1 2의 보수 기본

비트 패턴	부호 없는 값	부호 있는 값
0x00000000	0	0
0x00000001	1	1
0x7fffffff	2,147,483,647	2,147,483,647 (INT_MAX)
0x80000000	2,147,483,648	-2,147,483,648 (INT_MIN)
0xfffffffe	4,294,967,294	-2
0xffffffff	4,294,967,295	-1

-n의 비트 패턴은 2^32 - n이다. 예: -21 = 0x100000000 - 21 = **0xffffffeb**

5.2 우리 구현의 규칙

- **레지스터에는 항상 0 ~ 0xFFFFFFFF만 저장한다.**
  - 모든 쓰기는 `write_reg`(cpu.py:27-30)를 거치고, 거기서 `to_u32`로 자른다.
  - 예: `addi x1, x0, -1` → `0 + (-1) = -1` → `to_u32(-1)` = `0xffffffff`
- **즉시값은 해독할 때 부호 확장한 Python 정수(음수 가능)다.** `sign_extend`(bits.py:36)를 쓴다.
- **부호가 의미 있는 곳은 blt/bge뿐**이다. `to_signed32`(bits.py:25)로 해석한다.
  - add/sub/and/or/beq/bne는 비트 패턴 그대로 처리해도 결과가 같다(2의 보수의 장점).

5.3 lui + addi로 32비트 상수 만들기 (자주 나옴)

addi 즉시값은 **부호 확장**되므로 상수의 하위 12비트가 `0x800` 이상이면 음수로 더해진다. 그래서 lui 쪽을 1 올린다.

- 공식: `lui = (C + 0x800) >> 12`, `addi = C - (lui << 12)` (결과는 -2048 ~ 2047)
- 예 `0x12345abc`
  - 하위 `0xabc` ≥ `0x800`
  - lui `0x12346`, addi `0xabc - 0x1000 = -0x544 (-1348)`
  - 확인: `0x12346000 - 0x544 = 0x12345abc`
- 예 `0xdeadbeef`
  - 하위 `0xeef` ≥ `0x800`

    - lui `0xdeadc`, addi `-0x111`
- 예 `0x12345678`
    - 하위 `0x678` < `0x800`
    - lui `0x12345`, addi `0x678`

5.4 andi/ori 즉시값도 부호 확장 (cpu.py:70-73)

명령	imm 32비트	결과 (x1 = 0x12345678)
andi x, x1, -1	0xffffffff	0x12345678 (그대로)
andi x, x1, 0x7ff	0x000007ff	0x00000678
andi x, x1, -256	0xffffff00	0x12345600
ori x, x1, -2048	0xfffff800	0xfffffe78
ori x, x0, 0x555	0x00000555	0x00000555

5.5 부호 있는 비교 vs 부호 없는 비교

rs1	rs2	blt (부호, 우리 구현)	부호 없는 비교였다면
0xffffffff (-1)	1	**분기** (-1 < 1)	분기 안 함
1	0xffffffff	분기 안 함	분기
0x80000000 (MIN)	0x7fffffff (MAX)	**분기**	분기 안 함
0x80000000	1	분기 (bge는 안 함)	분기 안 함 (bge는 함)
5	5	안 함 (bge는 분기)	같음

5.6 Python 함정 (보고서 8장)

Python의 `&`, `>>`는 음수를 무한 길이의 2의 보수처럼 다룬다. 예: `-1 & 0xFF == 0xFF`

- 그래서 레지스터를 자르지 않아도 `sw`가 저장하는 바이트는 **우연히 맞는다.**
- 하지만 `beq`로 `-1`(자르지 않은 값)과 `0xffffffff`(lw로 읽은 값)를 비교하면 **같지 않다고 판단한다.**
- 변이 테스트가 이 빈틈을 찾아냈고, 테스트 08_wrap_compare를 추가해 막았다.

---

6. 메모리와 little-endian

- **읽기:** 주소 a의 4바이트 b0 b1 b2 b3 → `b0 | b1<<8 | b2<<16 | b3<<24` (memory.py:28)
- **쓰기:** 값 `0x12345678` → 주소 a부터 `78 56 34 12` (하위 바이트가 낮은 주소)
- **data.out 오프셋** = 주소 - 0x10000000. 예: 0x100003fc → 오프셋 1020(0x3fc)
- **유효 주소** = `to_u32(rs1 + imm)` (cpu.py:81, 84). 예: x5 = 0x10000400, `sw x3, -4(x5)` → 0x100003fc(마지막 워드)
- **data.bin이 1,024보다 짧으면** 나머지는 0이다. 예: data.bin = `11 22 33 44 55 66 77 88`이면
    - `lw 4` → `0x88776655`
    - `lw 8` → `0`
- **6바이트 data.bin** `01..06` → `lw 4` → `05 06 00 00` → `0x00000605`
- **음수 저장:** -2 → `fe ff ff ff`, -1 → `ff ff ff ff`
- **`lw x0, ...`도 메모리 접근은 한다.** 범위 검사를 하고, 결과만 버린다(write_reg에서 무시).

---

7. 분기와 PC

- **목표 주소 = 분기 명령어 자신의 주소 + imm** (cpu.py:90 등. `pc` = fetch_pc). PC+4 기준이 아니다.
- **imm 구하기:** 0x00040020의 beq가 0x00040008로 가려면 imm = 0x08 - 0x20 = **-24**
- **분기하지 않으면** `next_pc = pc + 4`(cpu.py:55)를 그대로 쓴다.
- **B형 범위:** -4096 ~ 4094 (짝수). 우리 프로그램에서는 명령어가 4바이트 정렬이라 4의 배수만 의미가 있다.
- **실행 명령어 수 세기:**
  - 루프 앞 명령 a개 + (반복당 명령 k개 × 반복 n번) + 루프 뒤 명령 b개
  - 예 (P3): 2 + 3×5 + 2 = 19

---

8. invalid 판별과 종료

8.1 판별 (decoder.py:102-148)

1. 필드를 추출한다(108~113행).
2. opcode가 6개 중 하나인지 확인한다. 아니면 148행 `return None`
3. opcode별 표에서 funct3(R형은 funct3+funct7)를 찾는다. 없으면 `return None`
4. 있으면 `Instruction(...)`을 반환한다.

opcode만 보면 생기는 대표 오답:

명령어	겉모습	실제 판정
mul	add와 opcode·funct3가 같다	funct7 = 0000001이므로 invalid
lb	lw와 opcode가 같다	funct3 = 000이므로 invalid
bltu	blt와 opcode가 같다	funct3 = 110이므로 invalid
sra	sub와 funct7이 같다	funct3 = 101이므로 invalid

8.2 fetch 당시 PC를 출력하는 근거 (명세 9.1 예시 문항)

1. `CPU.run` 39행에서 `fetch_pc = self.pc`로 fetch 주소를 따로 저장하고, 43행에서 `return fetch_pc, word`로 바로 그 값을 반환한다.
2. 우리 구현은 PC를 **미리 증가시키지 않는다.** `self.pc`는 46행에서 `execute`가 돌려준 다음 PC로만 바뀐다. invalid는 42~43행에서 execute 전에 반환되므로 PC가 바뀔 기회가 없다.
3. `sim.py:75`는 반환받은 `pc`를 `format_termination_line`(sim.py:29)에 그대로 넘겨 출력한다.

8.3 종료 순서

- **정상 종료:** `run` 반환 → `write_data_out`(sim.py:69, "wb", 1,024 B) → 표준 출력(sim.py:75) → 종료 코드 0
- **출력 형식:** `"PC 0x%08x invalid instruction 0x%08x\n"`
  - `%08x`는 소문자이고 8자리를 0으로 채운다.
  - 바이트로 출력하는 이유: Windows 텍스트 모드에서는 `\n`이 `\r\n`이 되기 때문이다.

8.4 실행 한도 (cpu.py:42-47)

- `executed` = 실행을 **마친** 정상 명령어 수
- 검사 순서: **42행 invalid 확인 → 44행 한도 확인.** 순서가 중요하다.
- **정확히 1,000,000개 실행 + 다음이 invalid → 정상 종료** (테스트 33)
  - 순서를 바꾸면 이 경우가 비정상 종료로 바뀐다.
- **1,000,000개 실행 + 다음이 지원 명령어 → 실행하지 않고 비정상 종료** (테스트 34·35)
  - 표준 오류에 메시지, 종료 코드 2, 표준 출력과 data.out 없음 (팀 결정)
- **작은 예 (한도를 바꿔 생각하기).** `addi x1,x0,4 / L: addi x1,x1,-1 / bne x1,x0,L / .word 0`은 정상 명령어 1 + 2×4 = **9개**를 실행한다.

한도	결과
9	정상 종료 (`PC 0x0004000c`)
8	8개 실행 후 다음 fetch = bne(0x00040008, 지원 명령어) → 비정상 종료

8.5 범위 밖 동작 (팀 결정, 종료 코드 3)

- text 밖 fetch (memory.py:53)
- 데이터 영역 밖 또는 비정렬 lw/sw (memory.py:61)
- text에 sw (데이터 영역이 아니므로 61행에서 걸림)

---

9. 바이너리 입출력 포인트

- 입력은 `"rb"` (memory.py:77)로 읽는다. 파일 안 바이트 `b7 00 00 10`은 문자열 "b7000010"이 아니라 값 `0x100000b7`이다.
- data.out은 `"wb"` (memory.py:87)로 쓴다.
  - 텍스트 모드로 쓰면 Windows에서 `0x0A` → `0x0D 0x0A`로 바뀌어 1,024 B를 넘는다.
  - 리눅스에서는 변환이 없어 우연히 맞는다.
- 표준 출력은 `sys.stdout.buffer` (sim.py:75)로 쓴다. `print()`를 쓰면 Windows에서 `\r\n`이 된다.
- data.out은 invalid를 만난 시점의 메모리다. 그 뒤 명령어의 효과는 들어가지 않는다.

---

10. 실행 결과 예측 연습 (정답은 시뮬레이터로 검산함)

모두 0x00040000부터 배치한다. data.bin은 특별한 말이 없으면 비어 있다(전부 0).

P1. data.bin = `78 56 34 12`

```
lui x1, 0x10000
lw x2, 0(x1)
addi x3, x2, -0x678
sw x3, 4(x1)
.word 0
```

- 정답: `PC 0x00040010 invalid instruction 0x00000000`
- data.out[4..7] = `00 50 34 12`
- 풀이: x2 = 0x12345678, x3 = 0x12345678 - 0x678 = 0x12345000 → little-endian으로 `00 50 34 12`

P2. (명세 9.1 변형)

```
addi x1, x0, -1
addi x2, x0, 1
lui x5, 0x10000
blt x1, x2, L
sw x1, 0(x5)
L: sw x2, 4(x5)
.word 0
```

- 정답: blt 분기(-1 < 1). [0..3] = `00 00 00 00`(sw x1 건너뜀), [4..7] = `01 00 00 00`
- `PC 0x00040018 ...`, 실행 5개

P3.

```
addi x1, x0, 5
addi x2, x0, 0
L: add x2, x2, x1
addi x1, x1, -1
bne x1, x0, L
lui x3, 0x10000
sw x2, 0(x3)
.word 0
```

- 정답: x2 = 5+4+3+2+1 = 15 → `0f 00 00 00`
- 실행 2 + 3×5 + 2 = 19개, `PC 0x0004001c ...`

P4.

```
lui x1, 0x10000
addi x2, x0, 0x12
sw x2, 0(x1)
mul x3, x2, x2
sw x2, 4(x1)
.word 0
```

- 정답: `PC 0x0004000c invalid instruction 0x022101b3`
- [0] = `12 00 00 00`, [4] = 0 (mul 뒤 sw는 실행 안 됨), 실행 3개
- mul 인코딩: `(1<<25) | (2<<20) | (2<<15) | (3<<7) | 0x33` = `0x022101b3`

P5.

```
addi x1, x0, 0x7ff
ori x2, x1, -2048
andi x3, x2, 0x0f0
andi x4, x1, -16
(x9 = 0x10000000에 x2, x3, x4 저장)
```

- 정답: x2 = 0x7ff | 0xfffff800 = **0xffffffff**, x3 = 0xf0, x4 = 0x7ff & 0xfffffff0 = **0x7f0**

P6. `lui x7, 0x12346; addi x7, x7, -1348`

- 정답: x7 = 0x12346000 - 0x544 = **0x12345abc** → 메모리 바이트 `bc 5a 34 12`

P7. data.bin = `11 22 33 44 55 66 77 88` (8 B)

```
lui x1, 0x10000
lw x2, 4(x1)
lw x3, 8(x1)
.word 0
```

- 정답: x2 = **0x88776655**, x3 = 0 (data.bin 뒤는 0으로 초기화), `PC 0x0004000c ...`

P8. `addi x0, x0, 5 / add x1, x0, x0 / addi x2, x0, 0 / .word 0`

- 정답: x0 = 0, x1 = 0. 레지스터가 전부 0인 채로 `PC 0x0004000c ...`

P9. `addi x1,x0,3 / addi x2,x0,5 / sub x3,x1,x2 / lui x9,0x10000 / sw x3,0(x9) / .word 0`

- 정답: x3 = -2 = **0xfffffffe** → `fe ff ff ff`, `PC 0x00040014 ...`

P10.

```
lui x1, 0x80000
addi x2, x0, 1
addi x10, x0, 0
bge x1, x2, S
addi x10, x10, 1
S: blt x1, x2, T
addi x10, x10, 2
T: (x10 저장)
```

- 정답: x1 = INT_MIN이다.
  - bge MIN ≥ 1 → 거짓 → +1
  - blt MIN < 1 → 참 → +2를 건너뜀
- x10 = **1**. 부호 없는 비교였다면 **2**다.

P11.

```
lui x1, 0x10000
addi x2, x0, 3
addi x3, x0, 0
L: addi x3, x3, 2
addi x2, x2, -1
blt x0, x2, L
sw x3, 0(x1)
sw x2, 4(x1)
.word 0
```

- 정답: 3번 반복 → x3 = 6, x2 = 0. 실행 3 + 3×3 + 2 = 14개, `PC 0x00040020 ...`

P12. 명세 예시에서 `sw x4, 0(x1)`을 `sw x4, 4(x1)`로 바꾸면?

- 정답: 출력은 같다(`PC 0x00040014 ... 0x00000000`).
- data.out은 [0..3] = 0, [4..7] = `0c 00 00 00`이 된다.

---

11. 오류 진단 (증상 → 원인 → 우리 코드에서 확인할 곳)

변이 테스트(`tests/mutation_test.py`)로 실제로 확인한 목록이다. 괄호 안은 그 버그를 잡는 테스트다.

증상	원인	우리 코드 위치
출력 PC가 항상 4 크다 (`0x00040018` 등)	fetch PC 대신 다음 PC 출력	cpu.py:39, 43 (`fetch_pc`)
`blt -1, 1`이 분기 안 함 / 14번 결과 0x99	부호 없는 비교	cpu.py:95, 98 (`to_signed32`)
정렬 결과에 음수가 뒤로 감	bge 부호 없는 비교	cpu.py:98
분기 위치가 4바이트씩 어긋남	목표를 PC+4+imm으로 계산	cpu.py:90, 93, 96, 99
오프셋이 2048 이상인 분기만 틀림	B형 imm11 누락	decoder.py:91
+2048이 -2048로 감	B형을 12비트로 부호 확장	decoder.py:94 (13이어야 함)
음수 오프셋 sw/lw만 틀림, 오프셋이 32 이상이면 틀림	S형 imm[4:0] 누락 / 부호 확장 누락	decoder.py:78-79
`addi x, x0, -1`이 0x00000fff	I형 부호 확장 누락	decoder.py:73
`andi x, y, -1`이 하위 12비트만 남김	andi/ori imm을 32비트로 확장 안 함	cpu.py:71, 73
x0이 0이 아님	x0 쓰기 무시 누락	cpu.py:29
beq가 -1과 0xffffffff를 다르다고 봄	레지스터 32비트 자르기 누락	cpu.py:30 (`to_u32`)
`mul`이 add처럼 실행됨	funct7 검사 누락	decoder.py:116, 42-47
`lb/sh/slli`가 실행됨	funct3 검사 누락	decoder.py:122, 128, 134
`bltu`가 blt로 실행됨	분기 funct3 표 오류	decoder.py:140, 51
lui 결과가 0 또는 이상한 값	lui 값을 한 번 더 `<< 12`	cpu.py:77 (`imm_u`가 이미 시프트 됨)
모든 lw 값이 바이트 순서가 뒤집힘 (`0x78563412`)	big-endian으로 읽음	memory.py:28
Windows에서만 data.out이 1,024 B보다 큼	텍스트 모드 쓰기	memory.py:87 (`"wb"`)
Windows에서만 표준 출력 비교 실패 (`\r\n`)	`print()` 사용	sim.py:75
data.out이 1,024 B보다 작음	data를 0으로 채우지 않음 / 범위 잘못	memory.py:46, 88
정확히 1,000,000개 후 invalid인데 비정상 종료	한도 검사를 invalid 검사보다 먼저 함	cpu.py:42-45 순서
1,000,001개째도 실행됨	`>=` 대신 `>`	cpu.py:44
대문자 16진수, 자릿수 부족	`%08X`, `%x`	sim.py:29

---

12. 작은 변경의 영향 분석

변경	결과	실패하는 우리 테스트
cpu.py:44 `>=` → `>`	1,000,001개까지 실행	34_limit_plus_one
cpu.py:42-45 두 if 순서 교환	정확히 1,000,000개 실행 후 invalid가 비정상 종료로 바뀜	33_limit_exact

변경	결과	실패하는 우리 테스트
cpu.py:43 `return self.pc, word`	**결과 같음.** 우리 구현은 PC를 미리 올리지 않아 `self.pc == fetch_pc`다. 다만 나중에 PC를 미리 올리도록 바꾸면 틀리므로 fetch_pc가 더 안전하다	없음
cpu.py:29 `if index != 0:` 삭제	x0에 값이 들어감	03_x0_hardwired
cpu.py:30 `to_u32` 삭제	레지스터에 음수·33비트 값이 들어감. sw는 우연히 맞고 beq/bne가 틀림	08_wrap_compare
cpu.py:95 `to_signed32` 제거	blt가 부호 없는 비교가 됨	14
cpu.py:71 `to_u32(imm)` → `imm & 0xFFF`	음수 imm의 andi/ori가 틀림	06_logic
cpu.py:55 `pc + 4` → `pc + 8`	모든 프로그램의 흐름이 망가짐	거의 전부
decoder.py:46 `(0b110, 0b0000000): "or"` 줄 삭제	`or`가 invalid로 처리되어 그 자리에서 종료	06_logic (or를 사용)
decoder.py:51에 `0b110: "blt"` 추가	bltu가 blt로 실행됨	29_invalid_bltu
memory.py:46 `bytearray(DATA_SIZE)` → `bytearray(data_bytes)`	data.bin이 짧으면 data.out도 짧아지고, 뒤쪽 lw는 IndexError	11, 12 등
memory.py:14 `DATA_BASE`를 다른 값으로	`lui x1, 0x10000` 기반 lw/sw가 범위 밖 → 종료 코드 3	데이터 쓰는 테스트 전부
cpu.py:7 `MAX_INSTRUCTIONS = 10`	정상 명령어를 11개 이상 실행하는 프로그램은 비정상 종료 (10개 이하는 그대로 통과)	04~10, 13, 14, 17~21, 33 (01·03·11·12·15·16·22~32는 10개 이하라 통과)
sim.py:75 → `print(...)`	Windows에서 `\r\n` → 바이트 비교 실패	모든 정상 종료 테스트 (Windows)
memory.py:87 `"wb"` → `"w"`	쓰기 에러(bytes를 텍스트 파일에 씀). 문자열로 바꾸면 Windows에서 0x0A가 변환됨	02, 11, 13 등
decoder.py:99 `& 0xFFFFF000` → `>> 12`	lui 값이 12비트 밀림	거의 전부

---

13. 테스트 설계 (서술형 대비)

원칙

- 기대값은 **시뮬레이터를 돌리지 않고 명령어 의미로 계산**한다(독립 기대 결과).
- 결과가 **관찰 가능**해야 한다. 레지스터는 출력되지 않으므로 반드시 `sw`로 data.out에 남긴다.
- 버그가 있을 때 **결과가 달라지도록** 설계한다. 예: 미리 0xAA로 채운 메모리에 0을 저장해 sw가 실제로 일어났는지 확인한다.
- **경계값**을 넣는다: imm -2048/2047, B형 +2048/-4096, 데이터 첫·마지막 워드, 한도 1,000,000 / 1,000,001
- **invalid 뒤에 효과가 보이는 명령어**를 둬서 "실행하지 않음"을 확인한다.

예시 답안 - "blt의 부호 비교를 검증하는 테스트를 설계하시오"

```
addi x1, x0, -1     # 0xffffffff
addi x2, x0, 1
lui  x5, 0x10000
addi x10, x0, 0
blt  x1, x2, L      # 부호 비교면 분기, 부호 없는 비교면 진행
ori  x10, x10, 1
L: sw x10, 0(x5)
.word 0
```

- 기대: x10 = 0 → data.out [0..3] = `00 00 00 00`, `PC 0x0004001c invalid instruction 0x00000000`
- 부호 없는 비교 버그가 있으면 `01 00 00 00`

"invalid 판별에서 funct7 검사를 검증" → `mul`(add와 opcode·funct3 같음)을 넣고, 그 뒤에 `sw`를 둔다. 기대: mul 주소와 `0x02...33` 값이 출력되고 sw 효과는 없다(우리 테스트 22).

"little-endian 검증" → data.bin = `78 56 34 12`, `lw` 뒤에 그대로 `sw`하거나 상수 연산 후 `sw` → 바이트 순서를 확인한다(우리 테스트 09).

"실행 한도 경계" → 1,000,000개 정확히 + invalid → 정상(33). 1,000,001개 필요 → 비정상(34).

우리 팀의 검증 계층 (보고서 7장): 손 계산 종단 간 40개 + 단위 30개 + 변이 29종 + Unicorn 참조 대조(손 계산 33개, 무작위 23,000개) + Docker 새 환경

---

14. 명세 9.1 예시 문항 모범 답안

Q1. 본인 팀의 little-endian 입력 처리를 설명하고, 바이트 78 56 34 12로 구성되는 값을 도출하시오.

- 입력 파일은 `memory.py:75-78 read_binary_file`에서 `open(path, "rb")`로 바이트 그대로 읽는다.
- `Memory.__init__`(42-48행)는 text를 `bytes`로, data를 1,024 B `bytearray`에 복사한다.
- 4바이트 값은 `read_word_le`(22-28행)가 만든다. 낮은 주소의 바이트 b0를 최하위에 두고 `b0 | (b1 << 8) | (b2 << 16) | (b3 << 24)`로 조립한다.
- fetch(56행)와 lw(68행)가 모두 이 함수를 쓴다.
- 78 56 34 12이면 b0 = 0x78, b1 = 0x56, b2 = 0x34, b3 = 0x12이므로 `0x78 + 0x5600 + 0x340000 + 0x12000000` = **0x12345678**이다.
- 반대로 sw는 `write_word_le`(31-36행)가 하위 바이트부터 저장한다.

Q2. 본인 팀의 blt 구현 위치를 제시하고 0xffffffff와 1을 비교할 때 분기 여부를 설명하시오.

- 해독: `decoder.py`의 `BRANCH_TABLE`(51행)에서 opcode 1100011·funct3 100을 blt로 판별한다. 즉시값은 `imm_b`(82-94행)로 13비트 부호 확장한다.
- 실행: `cpu.py:94-96 if to_signed32(rs1) < to_signed32(rs2): next_pc = pc + imm`
- 레지스터는 부호 없는 32비트 패턴으로 저장되어 있으므로 비교 직전에 `to_signed32`(bits.py:25-33)로 2의 보수 해석을 한다.
  - 0xffffffff는 bit 31이 1이므로 0xffffffff - 0x100000000 = **-1**
  - 1은 그대로 1이다.
- -1 < 1이 참이므로 **분기한다.** next_pc = blt 명령어 주소 + imm이다.
- 부호 없는 비교였다면 4294967295 < 1이 거짓이라 분기하지 않는다. 우리 테스트 14와 단위 테스트 `test_blt_spec_question`이 이를 확인한다.

Q3. 본인 팀에서 invalid instruction을 판별하는 과정을 설명하고, fetch 당시 PC를 출력하는 근거를 제시하시오.

- 판별: `decoder.decode`(102-148행)가 처리한다.
  - opcode [6:0], funct3 [14:12], funct7 [31:25]를 추출한다(108-113행).
  - opcode가 지원하는 6가지(0x33, 0x13, 0x03, 0x23, 0x63, 0x37)가 아니면 `None`(148행)
  - 지원 opcode라도 opcode별 표(42-51행)에 (funct3) 또는 (funct3, funct7) 조합이 없으면 `None`
  - 예: mul(funct7 = 0000001), lb(funct3 = 000), bltu(funct3 = 110)
- 종료:
  - `CPU.run`(cpu.py:42-43)이 `inst is None`이면 execute를 호출하지 않고 `(fetch_pc, word)`를 반환한다. 그 명령어와 이후 명령어는 실행되지 않는다.
  - `sim.main`은 data.out을 저장한 뒤(69행) 그 값을 출력한다(75행).
- fetch PC의 근거:
  - ① 39행에서 fetch 직전 PC를 `fetch_pc`에 따로 보존한다.
  - ② PC는 46행에서 execute가 끝난 뒤에만 바뀌므로, invalid에서는 PC가 증가하지 않는다.
  - ③ 반환값이 바로 그 `fetch_pc`다.
- 명세 예시에서 0x00040014의 0x00000000을 만나면 `PC 0x00040014 invalid instruction 0x00000000`이 나오고 0x00040018이 아니다.

---

15. 예상 문제 추가

1. **x0이 항상 0임을 어떻게 보장하나?**
   - `write_reg`(cpu.py:27-30)에서 `index != 0`일 때만 쓴다.
   - 읽을 때(cpu.py:52)는 regs[0]이 한 번도 바뀌지 않으므로 0이다.
2. **덧셈 오버플로를 어떻게 처리하나?**
   - 예외 없이 하위 32비트만 남긴다(`to_u32`, write_reg).
   - 예: 0x7fffffff + 1 = 0x80000000, 0xffffffff + 1 = 0
3. **lui x1, 0x10000의 동작은?**
   - `imm_u` = word & 0xFFFFF000 = 0x10000000을 그대로 rd에 쓴다.
   - 하위 12비트는 0이 된다(기존 값과 무관).
4. **sw의 유효 주소 계산과 저장 바이트는?**
   - addr = to_u32(rs1 + sext(imm_s))
   - `store_word`가 범위·정렬을 검사한 뒤 하위 바이트부터 4바이트를 저장한다.
5. **data.bin이 0바이트이면?**
   - 1,024 B가 모두 0인 데이터 영역에서 실행한다. data.out은 여전히 정확히 1,024 B다(테스트 11).
6. **text.bin의 마지막 4바이트가 0x00000000이어야 하는 이유는?**
   - 0x00000000은 opcode 0이라 invalid다. 프로그램 끝에서 반드시 정상 종료하게 하려는 보장이다.
7. **분기로 건너뛴 invalid 워드는?**
   - fetch되지 않으므로 종료를 일으키지 않는다. 판별은 적재가 아니라 fetch 시점에 한다(테스트 16).
8. **jal/ecall은?**
   - 지원 opcode가 아니므로 invalid로 종료한다. jal x0, 0(0x0000006f)이 실행됐다면 무한 루프였을 것이다(테스트 16, 30).
9. **명세가 정하지 않은 동작과 우리 결정은?**
   - 실행 한도 초과: 종료 코드 2, 표준 출력·data.out 없음
   - 범위·정렬 위반: 종료 코드 3

    - 크기 위반: 경고 후 진행
10. **왜 Python인가? 32비트 의미를 어떻게 지켰나?**
    - Python int는 크기 무제한이다. 그래서 저장 시 `to_u32`, 비교 시 `to_signed32`, 즉시값은 `sign_extend`로 명시적으로 처리했다.
11. **어셈블러(`tests/asm.py`)를 따로 만든 이유는?**
    - 해독기와 코드를 공유하면 같은 실수가 양쪽에 숨는다. 그래서 독립 인코더로 입력을 만들었다.
    - 명세 예시 바이트와 Unicorn 대조로 검증했다.
12. **변이 테스트란?**
    - 일부러 버그를 넣어 테스트가 실패하는지 본다.
    - 29종 모두 검출했다. 처음에 놓친 2종(32비트 자르기, `>` 한도) 때문에 테스트 08·34를 추가했다.
13. **참조 실행기를 어떻게 썼나?**
    - Unicorn Engine을 **테스트에만** 썼다(`tests/diff_ref_unicorn.py`).
    - Unicorn은 invalid에서 멈추지 않으므로, 종료 주소를 `until`로 주고 그 직전 상태를 비교했다.
    - 시뮬레이터 본체는 Unicorn을 쓰지 않는다(명세 8절 준수).
14. **표준 출력에 디버깅 정보를 넣으면?**
    - 바이트 비교가 실패한다. 진단은 표준 오류(`sim.error`)로만 쓰고, 추적은 별도 도구 `tools/trace.py`로 한다.
15. **0x00040000부터 명령어 n개가 있을 때 k번째 명령어 주소는?** 0x00040000 + 4(k-1)

---

16. 시험 전 체크리스트

- [ ] 인쇄본을 **최종 제출본**으로 만들었다(`study/build_pdfs.py` → `study/source_listing.pdf`, 줄 번호 포함). 메모를 쓰지 않았다.
- [ ] 2절 코드 지도의 위치를 인쇄본에서 바로 찾을 수 있다.
- [ ] 16진 → 2진 변환, R/I/S/B/U 필드 위치를 외웠다. 4.3~4.5 연습을 손으로 다시 풀었다.
- [ ] 2의 보수(-1, -2, -21, INT_MIN/MAX)와 부호 확장(0xfff → -1, 0x800 → -2048)을 바로 계산할 수 있다.
- [ ] lui+addi 상수 공식(+0x800)을 쓸 수 있다.
- [ ] little-endian 양방향 변환을 할 수 있다.
- [ ] 분기 목표 = 분기 명령어 주소 + imm, imm = 목표 - 현재를 계산할 수 있다.
- [ ] 실행 명령어 수 세기와 한도 경계(9개 예시)를 설명할 수 있다.
- [ ] 10절 P1~P12를 손으로 풀어 정답과 맞췄다.
- [ ] 11·12절 표의 각 행을 코드 위치와 함께 설명할 수 있다.
- [ ] 14절 모범 답안 3개를 자기 말로 쓸 수 있다.